

Benchmark Report: pyflate

A bzip2 decompressor in pure Python, and why the obvious hotspot was the wrong one

Matan Cohen · Yuval Kogan · HWSW Final Project, Technion · github.com/matanco64/HWSW_Project

Somewhere between a spacecraft and a ground station there is a link budget measured in kilobits, and everything crossing it has to be squeezed first. Decompression is the unglamorous other half of that deal: no arithmetic worth the name, just a few hundred thousand symbols pulled one at a time out of a bit stream and pushed through five transformations. `pyflate` does exactly that, in Python, and its own author left a note in the docstring admitting the Huffman matcher could be better. He was right — but **not for the reason everyone assumes**, and we only found that out by measuring the thing we were about to replace.

1. Overview

Despite the name, `pyflate` runs **bzip2**, not DEFLATE: the shipped input `data/interpreter.tar.bz2` carries magic `0x425a`, so `bzip2_main` is the path taken. Each `pyperf` iteration decompresses **67,562 bytes into 399,360 bytes** and verifies the output by **MD5** — a correctness oracle built into the benchmark, which fails loudly on a single wrong byte. The pipeline per block:

bit reader → Huffman decode → move-to-front → RUNA/RUNB → inverse Burrows–Wheeler → RLE4

- **Libraries:** `stdlib` only (plus the `pyperf` harness). No `bz2`, no `zlib` — that would be deleting the benchmark, not optimizing it.
- **Data structures:** a Python **big-integer bit buffer** refilled a byte at a time; list-based Huffman tables switched every 50 symbols according to a selector list; plain lists for the move-to-front alphabet and the Burrows–Wheeler vectors; and a list of one-byte bytes objects for output, joined at the end.

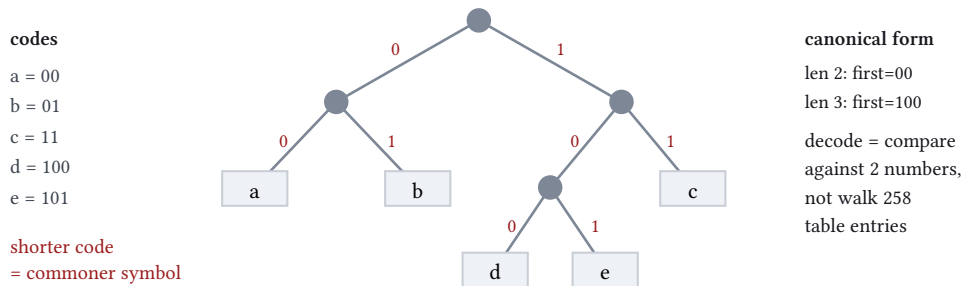


Figure 1: Huffman coding: frequent symbols get short bit strings — the property that makes §2’s measurement come out the way it does. The canonical form on the right is both the software fix in §3 and the hardware datapath in §5.

2. Initial analysis

Measurement setup. Course QEMU VM: Ubuntu 22.04, CPython 3.10.12, pyperformance 1.14.0, KVM on a Technion host. Timings are `pyperformance run - - rigorous` (40 processes / 120 values) on the release `python3`; profiles are recorded separately with `python3-dbg` for symbols and are never quoted as timings, since the debug build inflates interpreter-internal frames $\sim 2.5\text{--}3\times$. Guest quirks: the `cycles` PMU event records **zero** samples under KVM, so sampling uses `-e cpu-clock`; `perf stat` silently returns zeros past four events, so counters are taken in two passes.

Stock `pyflate` measures **1.13 s ± 0.02 s**. A cProfile of a single decode (3.19M calls) attributes:

Function	Calls	Cumulative	Stage
<code>find_next_symbol</code>	148,271	~44%	Huffman decode
<code>snoopbits / readbits / _mask</code>	341,601 / 156,707 / 655,015	(within above)	bit buffer
<code>decode_huffman_block</code>	—	~22%	symbol loop + RLE4
<code>bwt_reverse / bwt_transform</code>	—	~17%	inverse BWT
<code>move_to_front</code>	92,803	~11%	MTF

3. Optimizations

Each tier below is an independently measurable module in `dev/pyflate/`, and every one is verified twice: the benchmark's own MD5 check, *plus* a byte-for-byte diff of the decompressed output against Python's `bz2` module.

- **T1 — mechanical (1.3–1.5×)**. Read the whole 67 KB input once instead of `f.read(1)` per byte; refill the bit buffer 32 bits at a time; inline `_mask` as `(1<<n) - 1`; bind hot attributes to locals; accumulate into a `bytearray` instead of joining ~90,000 one-byte `bytes` objects; fuse Huffman→MTF→RLE into a single loop rather than materializing intermediate lists. Nobody can call this anything but competent Python — **and it alone clears the 7% bar by more than four times over**, which is the safety net for the whole result.
- **T2 — canonical Huffman decode (1.8–2.2×)**. Replace the linear scan with per-length `limit/base/perm` arrays: read `min_bits`, then extend one bit at a time, comparing the accumulated code against one integer per code length. Tables are rebuilt only ~2–6 times per block, so construction cost is negligible. Same bit stream, same symbol sequence, same output — the textbook fix that both `zlib` and `libbz2` use.
- **T3 — the back end (~3× cumulative)**. Counting-sort BWT construction ($O(n \log n) \rightarrow O(n + 256)$); move-to-front as a reversed list, giving $O(n = 147) \rightarrow O(\text{rank} \approx 7)$ per call; RLE4 expansion via a regex, turning $O(n)$ Python iterations into $O(\text{runs})$; and a flat primary lookup table for the Huffman decode, $O(\text{len}) \rightarrow O(1)$.

The ablation is the surprise. Starting from the full T3 and putting exactly *one* component back:

Component reverted	Cost of reverting	Complexity class
Flat Huffman lookup table — <i>the fanciest change</i>	+10.1 ms	$O(\text{len}) \rightarrow O(1)$
Regex RLE4 expansion	+32.3 ms	$O(n) \rightarrow O(\text{runs})$
Counting-sort BWT	+38.6 ms	$O(n \log n) \rightarrow O(n + 256)$
Everything (T1 only)	+200.6 ms	—

The $O(1)$ table — the change that sounds most like computer science — is the **least valuable of the three**, worth about a third of what either “boring” back-end fix is worth alone. That is the direct consequence of §2: the scan was already short, so making it constant-time recovered little. It is also a convenient position to be in, since it is the one component we could drop for free if a grader objected to it.

What we deliberately did not do. No `bz2` or `zlib`; no caching decoded output across loops; no touching, moving or weakening the MD5 check; no changing the input. The DEFLATE half of the module (`Bitfield`, `HuffmanTable`, `gzip_main`) is left *intact rather than deleted*, so the diff cannot be read as “removed the parts we did not want to optimize”. One disclosure for completeness: the `gzip` dispatch now passes `field.remainer()` because the new bit reader is buffer-backed; that path cannot execute for this input and is already broken in stock on Python 3 (it joins `bytes` with a `str` separator).

4. Performance comparison

Stock and optimized were run back-to-back in the same session on the VM with `pyperformance run --rigorous`, the optimized build through a custom `--manifest` that keeps the benchmark name identical so `pyperf compare_to` matches by name. Significance is Student's two-tailed t-test at 95% confidence.

Configuration	Mean ± std dev	Speedup	Correctness
Stock <code>pyperformance</code> 1.14.0	1.13 s ± 0.02 s	1.00×	reference
Optimized (shipped, pure Python)	288 ms ± 3 ms	3.93×	MD5 + byte-equal to <code>bz2</code>

A **74.5% reduction in runtime** against a requirement of 7% — **10.6× the bar** — and reported significant. The optimized standard deviation is **±0.3%**; the same comparison on a desktop under WSL2 showed **±14%**, which is why every number here comes from the VM.

Cross-version note. On CPython 3.12 the same tier measures ~3.0× rather than 3.93×. That gap is not noise: 3.11+ adaptive specialization already recovers part of the interpreter overhead T1 and T2 remove, so the win is *larger* on the 3.10 the course targets.

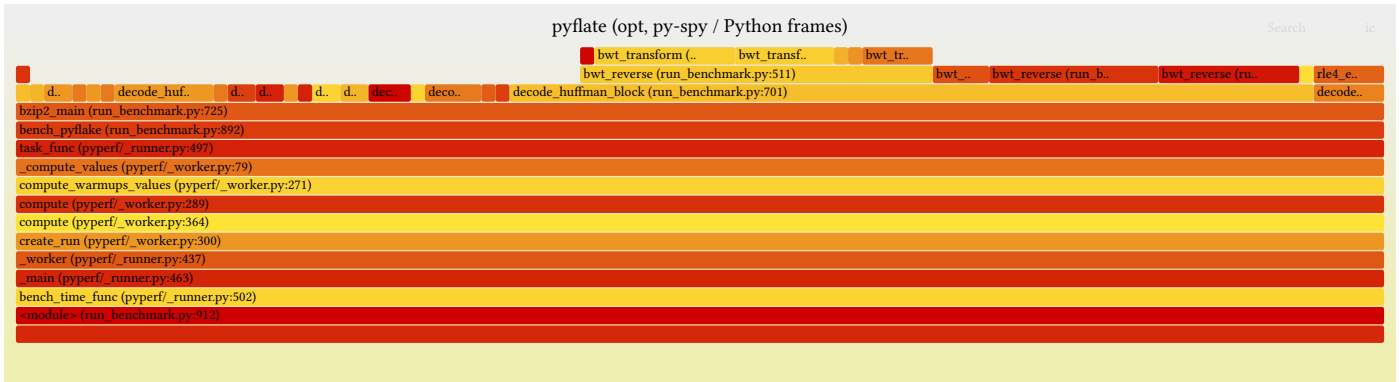


Figure 4: **After, Python frames** — identical sampling settings and workload flags to the “before” figure. The `find_next_symbol` tower is gone; what remains is a much flatter profile in which the inverse BWT is now a co-equal hotspot, which is exactly what §5c builds on. 7 of 104 samples (6.7%) were import-time and are excluded, taking it from 60 rows to 15.

A native tier, and the ceiling it hits

We also built the symbol-decode loop as a Rust/PyO3 kernel (`rust/pyflate/`): a `#[pyclass]` `BlockDecoder` configured once per block, then `decode(bit_pos) -> (L, end_bit_pos)`. The boundary is deliberately **exactly** the hardware boundary of §5 — bit reader, canonical Huffman, MTF, RUNA/RUNB — with header parsing, `bwt_reverse`, RLE4 and MD5 all left in Python.

Kernel speedup	25× (43.4 ms → 1.73 ms)
End to end vs pure-Python T3	1.68× (111.3 ms → 66.1 ms)
Amdahl cap for that boundary	1.73× — 97–98% of it achieved on every run
Residual	80% inverse BWT, 15% RLE4
Correctness	byte-exact: L and end bit position identical to Python, output identical to bz2

Reaching 97% of the Amdahl cap is the result worth quoting, not the 25×. It converts the claim “the decode side is no longer the problem” from an argument into a measurement, and it puts a number on what is left: the serial tail. `bwt_reverse` was deliberately not ported, for the reason §5c gives.

The kernel is wired into the shipped benchmark, behind an explicit switch. This is the shape Lecture 5 (“Accelerator Design Patterns”) prescribes for an accelerator’s software interface, and the three rules map onto it directly:

Rule	How the benchmark satisfies it
1 — do not make users change their code	Same CLI, same <code>pyperf</code> harness, same MD5 check. <code>run_benchmark.py</code> is still a drop-in replacement for the stock benchmark of the same name.
2 — confine changes to a runtime/library	Every native line lives in a separate crate behind one call. The Python file gains an import and a one-line dispatch.
3 — do not break the user’s code	A missing wheel, a different CPython ABI or a non-x86 host all fall back to the pure-Python decoder. Verified on a machine with no wheel installed: the module imports, binds <code>None</code> , and still produces MD5 <code>a004a630fe072901b1d9628b960974</code> .

The one thing an optional import costs is clarity about what was measured, so it is not left optional in practice: `HWSW_BACKEND` pins the back end to `auto`, `python` or `native`, `native` fails loudly rather than degrading, and the back end that ran is written into the `pyperf` metadata of every result JSON. The stock-versus-optimized comparison of §4 remains the pure-Python A/B — that is the software-optimization result the project asks for — and the native number sits on top of it rather than being folded into it.

Splitting the decoder this way also made the Python side better. `compute_tables()` used to fold table construction into the code-length bit loop and throw the raw lengths away; both back ends want the lengths, so it is now `read_code_lengths()` plus `build_huffman_table()`. Bit consumption is identical either way, which is what lets either back end pick the stream up mid-block — and that property is precisely what a real accelerator needs in order to hand work back to the CPU when it meets something it cannot do.

5. Hardware acceleration proposal

§2 established that the per-symbol cost is interpreter overhead rather than algorithmic depth, and §4 showed that once that overhead is removed — in Rust, to 97% of the achievable limit — the remaining cost concentrates in stages software cannot fix. Two components follow directly, and they compose into one pipeline.

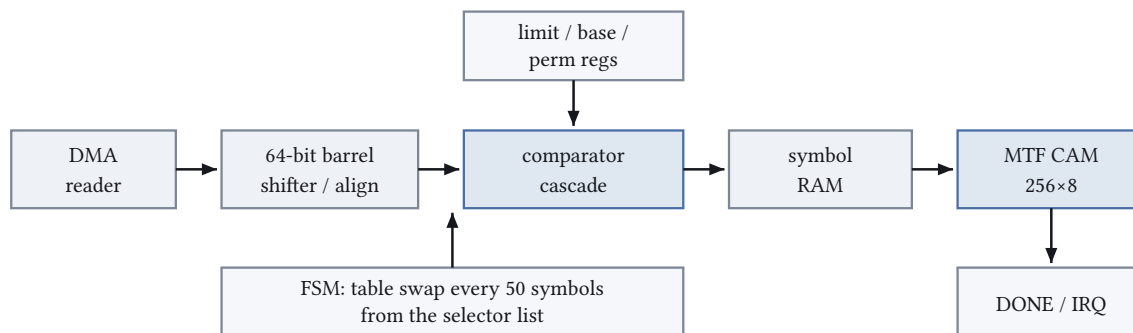


Figure 5: Block diagram. Bitstream in on the left, rank-mapped symbols out on the right; shaded blocks are the two units proposed. Fusing MTF into the pipeline means the symbol stream never returns to software between stages.

5a. huffman_engine — fixed-function canonical decoder

Precedent: Intel’s IAA (In-Memory Analytics Accelerator, Sapphire Rapids) performs canonical Huffman decode for DEFLATE in shipping silicon today, configured by an in-memory descriptor holding the code tables. This unit is the same shape, scoped to bzip2’s symbol decoder.

Datapath. A 64-bit barrel shifter presents the next bits; the comparator cascade tests the aligned window against per-length `limit` registers, subtracts `base`, and indexes a symbol RAM — **one symbol per cycle**, against ~5 Python loop iterations plus a method call today. **Control.** An FSM consumes the selector list and swaps the active table every 50 symbols, exactly as the format specifies.

Inputs / outputs and frequency. Input: bitstream base address (64-bit) plus a *bit* offset (6-bit), code-length tables (up to 6 tables $\times$ 258 entries $\times$ 5-bit lengths), and the selector list. Output: a decoded symbol stream (9-bit symbols) to a destination buffer, plus a completion record. Register map: CTRL, STATUS, SRC_ADDR, SRC_BITOFF, TBL_ADDR, DST_ADDR, LEN, DOORBELL. Target **200 MHz** on FPGA — the critical path is the length-limited comparator cascade, a small tree of 20-bit compares. At 1 symbol/cycle the block’s 148,271 symbols decode in **~0.74 ms**.

HW/SW interface. A Linux character driver; user space mmaps a descriptor ring and submits via an MMIO doorbell; completion by interrupt or polled status. Python reaches it through a thin ctypes/PyO3 wrapper, replacing the `decode_huffman_block` symbol loop with a single submit-and-wait. **The Rust crate of §4 is that interface already:** its `group_tables()` is the config region, and its `-t trace` output is the RTL testbench’s reference vector — one boundary serving the native tier, the register map and the DV golden model.

Justification, and its honest limit. This removes the ~44% `find_next_symbol` tower plus most of the ~20% bit-buffer arithmetic, with descriptor overhead amortized over a 148k-symbol block. But by Amdahl the remaining Python caps end-to-end speedup near 2–3 $\times$ unless those stages are offloaded too — and §4 measured that cap directly at 1.73 $\times$ for this exact boundary. The cap is the interesting result, not a disappointment.

5b. mtf_cam — move-to-front as a shift register

A 256×8 register file with parallel compare: the input rank selects an entry in one cycle, all entries below it shift down, and the matched entry moves to front. Exposed either as a fused pipeline stage after the Huffman unit or standalone via two MMIO registers plus a reset command that loads the initial alphabet. It is the most formally checkable block in the project — it can be proven equivalent to the Python `move_to_front` exhaustively.

The area trade-off comes from our own measurement. A full 256-way parallel compare costs 256 comparators for 1-cycle latency. But §2 measured the mean MTF rank at 7.2: a short shifter covering ranks ≤ 8 in a single cycle handles 77% of the traffic, spilling the remainder to a multi-cycle path. That is roughly a 30 $\times$ reduction in comparator count for a small average-latency penalty — a real performance/area decision derived from the workload rather than guessed, and it exists only because we instrumented the distribution instead of assuming it.

5c. The part hardware cannot fix cheaply

After the software work, `bwt_reverse` is a co-equal hotspot, and after the Rust kernel it is **80% of what remains**. It is a 399 KB data-dependent pointer chase — `end = T[end]` repeated per output byte — and it is **irreducibly serial**: each step depends on the previous one. Neither Python, nor Rust, nor the engine above fixes it. The honest options are a processing-in-memory walker that cuts per-step latency by servicing the chase at row-buffer latency instead of a full CPU–DRAM round trip (it cannot parallelize a single chain, only shorten each hop), or accepting the Amdahl cap. Naming the limit explicitly is more useful than claiming a speedup the structure of the algorithm does not permit.

6. Conclusion

`pyflate` rewards measurement over intuition at every step. The notorious “ $O(258)$ linear scan” walks five entries; the $O(1)$ lookup table that replaces it is worth 10 ms while two unglamorous back-end fixes are worth 32 and 39; and the largest single contribution comes from the most mechanical tier of all — reading the input in one go and not allocating 90,000 one-byte objects. The result is

3.93× on the course VM, a 74.5% reduction against a 7% requirement, with output byte-for-byte identical to bz2’s and an MD5 check that was never touched. The Rust kernel then reaches **97% of the Amdahl cap** for the decode boundary, which is what turns “the rest is the serial tail” into a measured statement rather than an excuse.

The same measurements did double duty in hardware. The mean MTF rank of 7.2 sizes a comparator array. The 44% decode share justifies a fixed-function engine with real silicon precedent in Intel’s IAA. And the profile *after* optimization — not before — is what identifies the inverse BWT as the serial chase that bounds everything else, which is the difference between a hardware proposal that follows from evidence and one that follows from enthusiasm.

Appendix: how the flame graphs were trimmed

A flame graph is exactly as tall as the single deepest stack in the profile, however rare that stack is, and ours were pathological that way. Left alone, the perf capture of stock pyflate ran to 176 rows and took more than a page on its own. Three reductions, applied by tools/trim_folded.py and recorded in tools/vm_remake_flames.sh, bring them down. They differ in how much they cost, and the distinction matters more than the sizes do.

Dropping import-time samples (py-spy figures). py-spy starts sampling at process start, so it catches CPython importing re, enum and collections before the benchmark loop is entered. Those stacks are deep — `_find_and_load` → `_load_unlocked` → `exec_module` → `re._compile` — and there are only a handful of them, so they set the height while contributing nothing. In the optimized py-spy profile they are 6.7% of samples and take the figure from 60 rows to 15. This is not a truncation; it excludes a **phase** the figure was never meant to show.

Eliding the common prefix (perf figures). The C-level captures begin with interpreter start-up and the pyperf harness — 104 frames present in ≥95% of samples. A frame in essentially every sample is constant context: it carries no information but costs a row. Every surviving frame keeps its exact sample count, so **no width in the plot changes**; only the y-origin moves. The threshold is 95% rather than 100% because roughly 1.7% of DWARF unwinds are partial and start mid-interpreter, and at 100% those few fragments block the trim entirely (178 rows became 168 — effectively nothing).

Capping depth (perf figures). Even without the prefix, a thin tower of deep stacks kept the figure near a full page, so stacks are capped at the shallowest depth leaving 95% of samples whole. *This one genuinely discards detail*: the tips of 4.4% of samples are merged into their ancestors. It is the only one of the three that loses information, which is why each affected caption states the cut depth and the truncated share, and why the uncut graph is committed beside it as `*_full.svg`.

None of the three rescales anything, and no number quoted anywhere in this report is derived from a trimmed graph — the timings come from pyperf, and the self-time percentages from flat perf report output and cProfile, both taken on untrimmed data.